

Document Loading Guide

1. Documents and sources

A document combines text with information about where that text came from. A support guide, product manual, policy, or research note can become a source for a retrieval system. Keeping a source name makes it possible to trace an answer back to the original material. Loading is the first stage: it reads source content into a predictable format. A loader does not decide whether a statement is correct or relevant to a question. Review the source before relying on it.

2. Plain text files

Text files are useful for notes, transcripts, and small knowledge bases. They contain readable content without page layout or embedded images. UTF-8 is a common encoding, but older exports may use another encoding. An encoding mismatch can cause an error or unreadable characters. TextLoader usually returns one document for the whole file. For a large file, split that document later. Keep headings in the text because headings help readers understand how paragraphs relate to each other.

Document Loading Guide

3. PDF documents

PDF files preserve a printed layout and can contain multiple pages, tables, images, and columns.

PyPDFLoader commonly produces a document for each page. Page metadata is useful when citing a source.

Text extraction may read columns in an unexpected order or lose table alignment. A scanned PDF contains an image of text and may require optical character recognition. Before using a PDF in a retrieval pipeline, inspect the extracted content and compare it with the original pages.

4. CSV records

CSV files store records in rows and fields in columns. A loader can turn each row into text with column names included. This makes it easier to interpret values when the row is retrieved later. A product catalog might contain a product name, category, stock level, and price. Row metadata helps locate the original record.

Delimiters and encodings vary between exports. Numeric values in loaded text are still text; loading alone does not calculate totals or enforce data types.

Document Loading Guide

5. Web pages

A web loader downloads HTML and extracts readable text. Navigation, footer links, and repeated banners can appear alongside the useful content. Some websites render their main content using JavaScript, which a basic HTML loader will not execute. Authentication may also prevent access. Save the URL in source metadata and check that the page text matches the intended article. Use a timeout for network requests. A failed request should be reported separately from a page that legitimately contains no useful content.

6. Wikipedia searches

WikipediaLoader searches for a topic and loads matching articles. A topic query is a search, not a guarantee of an exact page match. Similar names can return unrelated pages. Inspect each returned title and URL before using the article. Wikipedia pages change over time, and article length may be limited by loader settings. A blocked request can return HTML instead of the JSON expected by the API client. That failure does not establish whether the requested article exists.

Document Loading Guide

7. Chunking and retrieval

After loading, long documents are often split into smaller chunks. A small overlap can preserve context near a boundary. Very short chunks may lose meaning, while very long chunks can contain several unrelated topics. Preserve source and page metadata when splitting. Embeddings represent chunks for similarity search. A vector database stores those representations with the text. Retrieval chooses candidate chunks for a question; it does not prove that the chunks contain a complete or correct answer.

8. Checking a pipeline

Start with a few sources and known questions. Check document counts, empty content, broken characters, and unexpected metadata. For a PDF, compare a few extracted pages with the visible PDF. For CSV, inspect both the first and last rows. For web and Wikipedia sources, verify the source URL. Then evaluate whether retrieved chunks actually answer the question. If the information is absent, report that gap rather than inventing an answer. Keep sample data separate from personal documents and credentials.